

1 What is Cloud Computing?

✓ Definition

Cloud Computing is the delivery of **computing services**—like servers, storage, databases, networking, software, and analytics—**over the internet** (“the cloud”) instead of on local computers.

♦ Key Features

- On-demand access to resources
- Pay-as-you-go pricing
- Elasticity (scale up/down easily)
- Accessibility from anywhere

♦ Cloud Service Models

1. **IaaS (Infrastructure as a Service)** – e.g., virtual servers, storage (AWS EC2)
2. **PaaS (Platform as a Service)** – e.g., managed platforms for development (AWS Elastic Beanstalk)
3. **SaaS (Software as a Service)** – e.g., Google Workspace, Salesforce

♦ Deployment Models

- **Public Cloud** – Shared resources, e.g., AWS, Azure
- **Private Cloud** – Dedicated for a single organization
- **Hybrid Cloud** – Combination of public and private

2 Why We Use Cloud Computing

1. **Cost-Effective** – Pay only for what you use
2. **Scalable** – Add/remove resources instantly
3. **Global Access** – Access applications and data anywhere
4. **Reliability** – High uptime and backup services
5. **Faster Deployment** – Launch servers and apps quickly
6. **Security** – Advanced security features by cloud providers

⚡ 3 Why AWS is Often Preferred Over Azure & GCP

Feature	AWS	Azure	GCP
Market Share	Largest	2nd largest	Growing
Services	200+	100+	90+
Maturity	Very mature, 15+ years	Newer	Newer
Global Infrastructure	Most regions & availability zones	Fewer regions	Fewer regions
Community & Support	Large developer & partner ecosystem	Smaller	Smaller
Pricing	Flexible, many options	Flexible	Flexible but less mature

🌐 4 AWS Global Infrastructure

AWS uses a **highly distributed architecture** to provide **low-latency, reliable services worldwide**.

◆ Components

1. Regions

- Physical locations around the world
- Example: **us-east-1** (N. Virginia), **ap-south-1** (Mumbai)

2. Availability Zones (AZs)

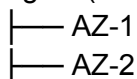
- Each region has **2+ isolated data centers**
- AZs provide **fault tolerance & high availability**

3. Edge Locations

- Used by **CloudFront (CDN)** to deliver content faster
- Example: Mumbai, Singapore, London

◆ Example Diagram (Conceptual)

Region (us-east-1)



└─ AZ-3
Edge Locations: NYC, LA, London...

1 Amazon S3 (Simple Storage Service)

✓ Definition

Amazon S3 is a **scalable object storage service** in the cloud for storing files (objects) like images, videos, documents, and backups.

◆ Key Features

- Unlimited storage
- Secure and durable (99.999999999% durability)
- Accessible from anywhere
- Cost-effective with multiple storage classes

2 Objects in S3

- **Object** = file stored in S3
- Components of an object:
 1. **Key** → file name
 2. **Value** → actual data/content
 3. **Metadata** → information about file
 4. **Version ID** (if versioning is enabled)

Example:

- File name: **image.png** → Key
- File data → Value
- Metadata → Content-Type: image/png

3 S3 Storage Classes

Class	Purpose	Cost	Use Case
S3 Standard	Frequently accessed	High	Web apps, content delivery
S3 Intelligent-Tiering	Auto-moves data to cheaper class	Medium	Unknown access patterns

S3 Standard-IA (Infrequent Access)	Less accessed	Lower	Backup, long-term storage
S3 One Zone-IA	Single AZ	Lower	Non-critical data
S3 Glacier	Archive	Very low	Long-term storage, months/years
S3 Glacier Deep Archive	Longest-term archive	Cheapest	Rarely accessed

4 Static Website Hosting on S3

Purpose

Host **static websites** (HTML, CSS, JS) directly from S3.

◆ Steps

1. Enable **Static Website Hosting** in S3 bucket settings
2. Upload **index.html** and **error.html**
3. Make objects **public**
4. Access website via **S3 endpoint URL**

5 IAM (Identity and Access Management)

Definition

IAM is used to **control access** to AWS resources securely.

◆ Components

1. **Users**
 - Individual AWS accounts for people or apps
 - Example: `alice@example.com`
2. **Groups**
 - Collection of users with **common permissions**
 - Example: `Developers`, `Admins`
3. **Roles**

- **Temporary permissions** assigned to AWS resources
- Used for EC2 instances or Lambda functions
- 4. **Policies**
 - JSON documents that **define permissions**
 - Types:
 - Managed Policy (AWS predefined)
 - Inline Policy (attached to user/group/role)
- 5. **MFA (Multi-Factor Authentication)**
 - Adds **extra layer of security**
 - Example: OTP from mobile app along with password

◆ **Example of IAM Usage**

- **User:** Bob
- **Group:** Developers → attach **S3ReadOnly** policy
- **Role:** EC2 instance can access S3 → attach **S3FullAccess** role
- **MFA:** Bob's account requires OTP login

1 Amazon EC2 (Elastic Compute Cloud)

✓ **Definition**

Amazon EC2 is a **scalable virtual server** service in AWS that lets you **run applications in the cloud**.

◆ **Key Features**

- On-demand virtual machines
- Choose OS, CPU, memory
- Pay-as-you-go
- Auto Scaling support
- Secure and highly available

◆ **2 AMI (Amazon Machine Image)**

✓ **Definition**

AMI is a **template** containing:

- Operating system (Linux/Windows)
- Software & configuration
- Launch configuration for EC2

♦ Example

- Use Amazon Linux 2 AMI → Launch EC2 instance
- Use custom AMI → Pre-installed apps

♦ **3 EC2 Instance Family**

Family	Purpose	Example
T series	General purpose, burstable	t2.micro, t3.small
M series	General purpose	m5.large, m6i.large
C series	Compute-optimized	c5.large
R series	Memory-optimized	r5.large
G/P series	GPU	g4dn.xlarge, p3.2xlarge

♦ **4 Key Pair**

✓ Definition

- **Key pair** = public/private keys for **secure SSH access** to EC2
- Private key **.pem** is downloaded and used to connect

♦ Example

```
ssh -i mykey.pem ec2-user@ec2-public-ip
```

♦ **5 Security Group (SG)**

✓ Definition

Virtual firewall for EC2 controlling **inbound/outbound traffic**.

◆ Example Rules

Direction	Protocol	Port	Source
Inbound	TCP	22	0.0.0.0/0 (SSH)
Inbound	TCP	80	0.0.0.0/0 (HTTP)
Outbound	All	All	0.0.0.0/0

◆ 6 EBS (Elastic Block Store)

✓ Definition

Persistent storage for EC2 instances.

◆ Features

- Volume types: SSD, Provisioned IOPS, HDD
- Can detach and attach to other EC2
- Data persists after instance termination (if configured)

7 VPC (Virtual Private Cloud)

✓ Definition

- **Isolated network** in AWS where EC2 and other resources run
- You control subnets, IP ranges, routing, gateways

◆ 8 Internet Gateway (IG)

✓ Definition

- Connects **VPC to the internet**
- Required for public subnets to access internet

◆ 9 NAT Gateway

✓ Definition

- Allows **instances in private subnet** to access internet
- **Private instances can download updates** without being publicly accessible

◆ 10 Public & Private Subnets

Subnet Type	Definition	Internet Access
Public	Directly accessible via Internet Gateway	Yes
Private	Only accessible via NAT Gateway	No (directly)

◆ 11 Route Table

✓ Definition

- Controls **network traffic routing** in a subnet

◆ Example

Destination	Target
0.0.0.0/0	IGW (public subnet)
0.0.0.0/0	NAT Gateway (private subnet)

CI/CD on AWS – Detailed Notes

1 Load Balancer (Elastic Load Balancer – ELB)

Purpose:

Distribute incoming traffic across multiple instances to ensure high availability and fault tolerance.

Types:

Type	Use Case
ALB (Application Load Balancer)	HTTP/HTTPS traffic, advanced routing (host/path-based)
NLB (Network Load Balancer)	High-performance TCP traffic, low latency
CLB (Classic Load Balancer)	Legacy apps, basic load balancing

Key Features:

- Health checks to detect unhealthy instances.
- SSL/TLS termination.
- Sticky sessions (optional).
- Integration with Auto Scaling.

AWS Console / CLI Commands:

- Create ALB: `aws elbv2 create-load-balancer --name my-alb ...`
- Configure Target Group: `aws elbv2 create-target-group ...`

Importance:

Ensures your application can handle traffic efficiently and failover automatically.

Auto Scaling

Purpose:

Automatically adjusts the number of EC2 instances in response to demand to maintain performance and cost efficiency.

Components:

- **Launch Configuration / Launch Template** – Defines AMI, instance type, security groups.
- **Scaling Policy** – Determines when to scale in/out (based on CPU, memory, custom metrics).
- **Health Checks** – Works with ELB to terminate unhealthy instances and launch new ones.

Example:

- Scale out if CPU > 70% for 5 minutes.
- Scale in if CPU < 30% for 10 minutes.

Importance:

Keeps the application available under load and reduces costs when traffic decreases.

3 CodeCommit

Purpose:

AWS's fully-managed Git repository service for source control.

Features:

- Private Git repositories.
- Integrated with IAM for access control.
- Supports pull requests and branching.
- Encryption at rest.

Usage:

Clone repository

```
git clone https://git-codecommit.us-east-1.amazonaws.com/v1/repos/MyRepo
```

Push changes

```
git add .
```

```
git commit -m "Initial commit"
```

```
git push origin main
```

Importance:

Keeps source code version-controlled and integrates directly with CodePipeline for automated CI/CD.

4 CodeBuild

Purpose:

Fully managed build service that compiles code, runs tests, and produces artifacts.

Key Points:

- Build environments for multiple languages (Java, Python, Node.js, etc.).

- Can pull source from CodeCommit, GitHub, or S3.
- Outputs build artifacts to S3 or CodePipeline.

Build Spec Example (**buildspec.yml**):

version: 0.2

phases:

install:

runtime-versions:

python: 3.9

commands:

- pip install -r requirements.txt

build:

commands:

- python -m unittest discover

artifacts:

files:

- '**/*'

Importance:

Automates compilation, testing, and artifact creation.

5 CodeDeploy

Purpose:

Automates deployment of applications to EC2, Lambda, or ECS.

Deployment Types:

- **In-place deployment:** Updates instances one by one.
- **Blue/Green deployment:** Creates new instances for the new version, reduces downtime.

Key Concepts:

- **AppSpec file (**appspec.yml**)** defines deployment instructions.
- Hooks: PreInstall, Install, PostInstall, ApplicationStart.

Example **appspec.yml**:

version: 0.0

os: linux

files:

- source: /build/
destination: /var/www/html/

hooks:

AfterInstall:

- location: scripts/restart_server.sh
timeout: 300
runas: root

Importance:

Automates deployments with minimal downtime and rollback options.

6 CodePipeline

Purpose:

Automates the CI/CD workflow by orchestrating CodeCommit → CodeBuild → CodeDeploy.

Pipeline Stages:

Stage	AWS Service
Source	CodeCommit / GitHub
Build	CodeBuild
Deploy	CodeDeploy / ECS / Lambda

Features:

- Continuous delivery pipeline.
- Manual approval stage optional.
- Integration with CloudWatch for monitoring.
- Triggers on code commit or schedule.

Example Flow:

1. Developer pushes code to CodeCommit.
2. CodePipeline triggers CodeBuild to run tests and build artifacts.
3. Successful build triggers CodeDeploy to deploy to EC2 instances behind ELB.
4. Auto Scaling ensures capacity for new traffic.

Importance:

Centralizes CI/CD automation with minimal manual intervention.

7 Route 53 (DNS Service)

Purpose:

AWS's highly available and scalable DNS service.

Key Features:

- Domain registration and management.
- Routing policies: simple, weighted, latency-based, failover.
- Health checks integrated with ELB and endpoints.

Example Use Cases:

- Route traffic to multiple regions.
- Automatic failover if ELB becomes unhealthy.

Importance:

Ensures users reach healthy, highly available endpoints.

8 Artifact Management

Purpose:

Stores build outputs for deployment.

AWS Options:

- **S3:** Common for storing `.zip`, `.tar.gz`, or Docker images.
- **CodeBuild/CodePipeline Artifacts:** Automatically stored in S3 buckets.

Best Practices:

- Version artifacts with commit ID or build number.
- Encrypt artifacts for security.
- Integrate with CodeDeploy for automated deployment.

Importance:

Keeps a traceable, versioned history of build outputs; enables rollback.

9 Typical AWS CI/CD Workflow

Developer pushes code → CodeCommit
↓
CodePipeline detects changes → triggers CodeBuild
↓
CodeBuild builds & tests → generates artifact → stores in S3
↓
CodeDeploy deploys artifact → updates EC2 instances behind ELB
↓
Auto Scaling adjusts capacity if needed
↓
Route 53 ensures traffic is routed to healthy endpoints

Key Benefits:

- Fully automated CI/CD workflow.
- Zero or minimal downtime.
- Scalable and fault-tolerant architecture.
- Easy rollback and versioning.

1 Amazon EKS (Elastic Kubernetes Service)

Purpose:

Managed Kubernetes service that runs Kubernetes clusters on AWS without managing the control plane.

Key Concepts:

- **Cluster:** Collection of worker nodes managed by Kubernetes.
- **Node Group:** EC2 instances running container workloads.
- **Control Plane:** Fully managed by AWS (highly available).

Key Features:

- Auto-scaling of pods via Cluster Autoscaler.
- Integration with **ELB** for service load balancing.
- Supports **Fargate** for serverless container workloads.

Commands / Usage:

- Create cluster:

```
aws eks create-cluster --name my-cluster --role-arn <IAM-Role-ARN> --resources-vpc-config  
subnetIds=<subnet-ids>,securityGroupIds=<sg-ids>
```

Importance:

Runs containerized applications with Kubernetes orchestration without managing the control plane.

2 Amazon ECS (Elastic Container Service)

Purpose:

Fully managed container orchestration service to run Docker containers on AWS.

Modes:

Mode	Description
EC2 launch type	Run containers on EC2 instances you manage
Fargate launch type	Serverless, AWS manages the compute layer

Key Concepts:

- **Cluster:** Logical grouping of EC2 or Fargate instances.
- **Task Definition:** Blueprint for running a container (image, CPU, memory, env variables).
- **Service:** Ensures desired number of tasks are running.

Importance:

Simplifies container deployment and scaling; tightly integrated with other AWS services.

3 Amazon ECR (Elastic Container Registry)

Purpose:

Managed Docker container registry to store, version, and deploy container images.

Features:

- Fully managed, secure, and scalable.
- Integrates with ECS, EKS, and CodePipeline.
- Supports image scanning for vulnerabilities.

Usage:

Authenticate Docker to ECR

```
aws ecr get-login-password --region us-east-1 | docker login --username AWS --password-stdin  
<account>.dkr.ecr.us-east-1.amazonaws.com
```

Build and push image

```
docker build -t my-app .
```

```
docker tag my-app:latest <account>.dkr.ecr.us-east-1.amazonaws.com/my-app:latest
```

```
docker push <account>.dkr.ecr.us-east-1.amazonaws.com/my-app:latest
```

Importance:

Provides a secure, scalable place to store container images for deployment.

4 AWS Fargate

Purpose:

Serverless compute engine for containers. No need to manage servers.

Key Features:

- Runs containers without managing EC2 instances.
- Integrated with ECS and EKS.
- Automatically scales based on task requirements.

Advantages:

- Pay only for resources used.

- Reduces operational overhead.
 - Ideal for microservices and event-driven workloads.
-

5 AWS CloudFormation

Purpose:

Infrastructure as Code (IaC) service for deploying AWS resources in an automated, repeatable manner.

Key Concepts:

- **Template:** JSON/YAML file describing AWS resources.
- **Stack:** A running collection of resources from a template.
- **Change Sets:** Preview changes before applying.

Example Template Snippet (ECS Service):

Resources:

MyCluster:

Type: AWS::ECS::Cluster

Properties:

ClusterName: my-cluster

MyTaskDefinition:

Type: AWS::ECS::TaskDefinition

Properties:

ContainerDefinitions:

- Name: my-app

Image: <account>.dkr.ecr.us-east-1.amazonaws.com/my-app:latest

Memory: 512

Cpu: 256

Importance:

Enables repeatable deployments and version-controlled infrastructure.

6 Amazon CloudWatch

Purpose:

Monitoring and observability service for AWS resources and applications.

Features:

- **Metrics:** CPU, memory, disk, network, custom metrics.
- **Alarms:** Trigger notifications or automated actions.
- **Logs:** Centralized logging for EC2, ECS, Lambda, EKS pods.
- **Events:** Respond to resource changes or scheduled events.

Example:

- Set alarm if ECS CPU > 80% for 5 minutes.
- Stream logs from ECS containers to CloudWatch Logs.

Importance:

Provides real-time insights into system health and performance.

7 AWS CloudTrail

Purpose:

Records AWS API calls for governance, compliance, and auditing.

Features:

- Tracks **who did what, when, and from where**.
- Supports S3, Lambda, ECS, EKS, CloudFormation events.
- Can trigger CloudWatch events for automated alerting.

Example Use Case:

- Detect unauthorized IAM role changes.
- Monitor ECS task changes for compliance.

Importance:

Essential for security audits, incident investigation, and compliance requirements.

8 Typical AWS Container CI/CD + Monitoring Workflow

CodeCommit / GitHub



CodeBuild (build & push Docker image to ECR)



ECS / EKS deploy (with Fargate or EC2)



CloudFormation manages infrastructure



Load Balancer distributes traffic



Auto Scaling adjusts container capacity



CloudWatch monitors metrics and logs



CloudTrail records API calls for auditing

Key Benefits:

- Fully automated container deployment.
- Scalable, highly available infrastructure.
- Continuous monitoring and auditing for compliance.
- Reduced operational overhead with Fargate and CloudFormation.